100Plante

Détection de maladies sur feuilles de plantes par transfert d'apprentissage

Dossier final du projet

Dépôt du code : https://github.com/POUKONE/100Plante
Application : https://100-plante.vercel.app
Date du document : 9 octobre 2026

Table des matières

1. Contexte et problématique . . . 3
1.1 Contexte . . . 3
1.2 Problématique . . . 3
1.3 Cadre de réalisation . . . 3
2. Objectifs et périmètre . . . 4
2.1 Objectifs . . . 4
2.2 Périmètre . . . 4
2.3 Arbitrages de périmètre en cours de projet . . . 4
3. Besoins couverts . . . 5
4. Technologies et outils . . . 5
5. Méthodologie de travail . . . 6
6. Réalisation . . . 7
6.1 Données . . . 7
6.2 Baseline . . . 7
6.3 Transfert d'apprentissage . . . 7
6.4 Évaluation . . . 8
6.5 API . . . 8
6.6 Interface . . . 8
6.7 Mise en ligne . . . 8
7. Choix techniques et justifications . . . 9
8. Résultats . . . 9
8.1 Performances sur le jeu de test . . . 9
8.2 Critères d'acceptation et objectifs . . . 10
8.3 Analyse des erreurs . . . 11
8.4 État au 9 octobre 2026 . . . 12
9. Difficultés rencontrées et solutions . . . 13
10. Limites du travail . . . 14
11. Perspectives d'amélioration . . . 15
12. Conclusion . . . 15

1. Contexte et problématique

1.1 Contexte

Les maladies foliaires sont une cause de pertes de rendement agricole. Leur repérage repose le plus souvent sur l'observation visuelle d'un agronome ou de l'exploitant : la méthode est lente, difficile à appliquer à grande échelle et sujette à l'erreur humaine, en particulier là où l'expertise phytosanitaire est rare. Ce constat ouvre le cahier des charges du projet (version 1.1, datée du 2 août 2026), qui demande un système capable d'identifier, à partir d'une photographie de feuille, la présence et le type d'une maladie.

Le projet s'appuie sur PlantVillage, un jeu de données public d'environ 54 300 images de feuilles saines et malades, photographiées en laboratoire sur fond neutre. Il couvre 14 espèces cultivées (pomme, myrtille, cerise, maïs, raisin, orange, pêche, poivron, pomme de terre, framboise, soja, courge, fraise, tomate) et 38 classes, chaque classe correspondant à une espèce dans un état donné : saine, ou atteinte d'une maladie précise.

1.2 Problématique

La question à laquelle répond le projet est la suivante : comment identifier, à partir d'une seule photo de feuille, l'état sanitaire d'une plante parmi 38 possibilités, avec un niveau de confiance exploitable, et présenter ce résultat de façon compréhensible à une personne qui n'est pas spécialiste ?

Deux difficultés apparaissent dès le cadrage. Les classes sont très inégalement représentées dans le jeu de données, ce qui expose le modèle à négliger les maladies rares. Par ailleurs, une exactitude obtenue sur des images de laboratoire ne préjuge pas de celle que l'on obtiendrait sur des photos prises au champ ; le cahier des charges le signale comme un risque à documenter.

1.3 Cadre de réalisation

Le projet est individuel. Sa partie technique (données, modèle, API, interface, déploiement) a été réalisée entre le 2 août 2026 en soirée et le 3 août 2026 en matinée, d'après les horodatages des commits et des journaux d'exécution ; le détail figure dans le compte rendu chronologique. Le travail a été mené sur un seul poste, un Mac à puce Apple M1 avec 8 Go de mémoire.

Le développement s'est fait en dialogue avec un assistant de programmation fondé sur un modèle de langage (Claude Code, d'Anthropic). L'assistant a rédigé et exécuté une partie importante du code et des vérifications, sous la direction de l'auteur. L'auteur a arbitré les choix de périmètre (section 2.3) et a lui-même exécuté les scripts de préparation des données, d'extraction de caractéristiques, d'entraînement et d'évaluation, en communiquant leurs sorties. Les chiffres de ce dossier proviennent de l'exécution de ces scripts.

2. Objectifs et périmètre

2.1 Objectifs

L'objectif général est de concevoir, d'entraîner et d'évaluer un modèle de classification d'images qui reconnaît l'état sanitaire d'une feuille, saine ou atteinte d'une maladie précise, en s'appuyant sur le transfert d'apprentissage. Le cahier des charges en déduit cinq objectifs spécifiques :

- atteindre une exactitude globale supérieure à 90 % sur un jeu de test indépendant ;
- fournir, pour chaque prédiction, un niveau de confiance et les trois classes les plus probables ;
- produire un modèle assez léger pour un usage mobile ou embarqué (objectif indicatif : moins de 20 Mo après conversion) ;
- livrer une interface de démonstration permettant de tester le modèle sur une image importée ;
- documenter les limites du modèle, notamment sur des images prises hors laboratoire.

2.2 Périmètre

Sont inclus : la classification sur les 38 classes de PlantVillage, l'entraînement par transfert d'apprentissage à partir d'un modèle pré-entraîné sur ImageNet, l'évaluation quantitative (exactitude, précision, rappel, F1-score, matrice de confusion), une interface de démonstration et la documentation.

Sont exclus : les cultures absentes de PlantVillage, le déploiement sur une flotte de capteurs ou de drones, la recommandation automatisée de traitement phytosanitaire et la collecte de nouvelles données de terrain.

2.3 Arbitrages de périmètre en cours de projet

- **Pas de conduite à tenir dans le diagnostic.** L'auteur a demandé que le diagnostic indique aussi « ce qu'il faut faire ». Cette demande entrait en conflit avec la section 3.2 du cahier des charges (« le système identifie la maladie, il ne prescrit pas de solution »), et la section 12 n'envisage un tel module qu'avec un expert agronome. L'auteur a retenu l'option la plus restreinte : décrire la maladie (agent pathogène, symptômes, mode de propagation) sans recommandation de traitement ni bonne pratique générale.
- **Référence externe reprise en partie.** Le descriptif d'une application de diagnostic de plantes (« SOS Plantes ») a servi d'inspiration. Ont été repris le choix entre photo et fichier et une section « Comment ça marche ». Ont été écartés les jauges d'eau, de lumière et de nutriments (le modèle ne mesure pas ces grandeurs), les marqueurs de zones atteintes (ils relèvent de la segmentation, section 12 du cahier) et les boutons de téléchargement d'application mobile (hors périmètre).
- **Nom de l'application.** Le nom « 100Plante » a remplacé l'intitulé initial « Leaf Disease Detection » pendant la phase d'interface.

- **Modèle de base.** MobileNetV2 a été retenu ; EfficientNetB0, également admis par le cahier des charges, n'a pas été implémenté.

3. Besoins couverts

Tableau 1. Exigences fonctionnelles du cahier des charges et réponses apportées.

Exigence du cahier des charges	Réponse apportée
Entrée : image JPEG ou PNG	Route `POST /api/predict` ; types `image/jpeg` et `image/png` acceptés, 10 Mo au maximum.
Sortie : classe prédite (espèce et état) avec un score de confiance	Champ `predicted` de la réponse : libellé brut, espèce, état, confiance entre 0 et 1.
Trois classes les plus probables lorsque la confiance est faible (seuil indicatif de 70 %)	Champ `top_k` toujours renvoyé ; indicateur `low_confidence` vrai sous 70 % ; l'interface en tient compte dans son message.
Traitement d'une image unique via une interface de démonstration	Pages `/demo` (import et analyse) et `/resultat` (diagnostic).
Rapport d'évaluation exportable après entraînement	Dossier `backend/reports` : rapport Markdown, métriques JSON, matrice de confusion en CSV et PNG, rapport de classification par classe.

Trois besoins se sont ajoutés en cours de route : l'affichage des 38 classes en français, une fiche descriptive de la maladie détectée, et une indication de fiabilité propre à chaque classe, fondée sur l'évaluation sur jeu de test. L'application a de plus été adaptée aux écrans de téléphone, de tablette et d'ordinateur.

4. Technologies et outils

Tableau 2. Technologies et outils utilisés.

Domaine	Outil	Version	Usage
Langage	Python (pyenv, venv)	3.11.6	Backend, scripts de données et d'entraînement
Apprentissage profond	TensorFlow / Keras	2.18.0	MobileNetV2, entraînement, inférence
Accélération locale	tensorflow-metal	non relevée	GPU Apple, entraînement en local uniquement
Apprentissage classique	scikit-learn	1.6.1	Random Forest, pondération des classes, métriques
Caractéristiques de la baseline	scikit-image	0.25.0	Histogramme de gradients orientés (HOG)
Données	TensorFlow Datasets	4.9.10	Téléchargement de PlantVillage
Images et calcul	Pillow, NumPy	11.1.0, 1.26.4	Décodage, redimensionnement, tableaux
API	FastAPI, Uvicorn, Pydantic Settings	0.115.6, 0.34.0, 2.7.1	Routes, validation, configuration

Domaine	Outil	Version	Usage
Tests	pytest, httpx	8.3.4, 0.28.1	Tests de l'API
Graphiques	matplotlib	3.10.0	Matrice de confusion
Interface	Next.js, React, TypeScript	16.2.12, 19.2.4, 5	Application web, CSS Modules, polices IBM Plex
Conteneur	Docker	28.5.1	Image de l'API
Hébergement	Fly.io (flyctl), Vercel	0.4.77	API (Fly.io), interface (Vercel)
Versionnement	Git, GitHub	–	Dépôt public POUKONE/100Plante

5. Méthodologie de travail

Le travail suit les six phases du cahier des charges : préparation des données, baseline, transfert d'apprentissage, évaluation, direction artistique et démonstrateur, documentation et livraison. Les cinq premières ont été menées dans cet ordre. La mise en ligne, que le cahier ne prévoit pas comme phase distincte, s'est ajoutée à la cinquième ; la sixième est couverte par le présent dossier et la documentation technique.

- **Baseline avant modèle.** Un classifieur simple (Random Forest sur couleur et HOG) fixe le seuil que le modèle de transfert doit dépasser strictement.
- **Jeu de test réservé.** Le jeu de validation sert au suivi et à l'arrêt anticipé ; le jeu de test n'est utilisé que pour l'évaluation finale.
- **Décisions consignées dans le code.** Les choix de méthode (pondération des classes, normalisation, ordre des classes) sont écrits dans les docstrings des scripts.
- **Maquette avant code pour l'interface.** Trois maquettes HTML (accueil, démonstration, résultat) ont été validées par l'auteur avant la construction de l'application Next.js, conformément à la section 7.7 du cahier des charges.
- **Vérification en conditions réelles.** Chaque étape a été contrôlée en exécutant le code : tests automatisés, appels de l'API avec de vraies images, parcours complet dans un navigateur, construction et lancement du conteneur, essais sur l'hébergement final.

Pour la préparation des données, l'auteur avait demandé à y participer pour l'essentiel (environ 60 %, contre 40 % pour l'assistant). Concrètement, l'assistant écrivait et expliquait les scripts ; l'auteur les lançait, lisait les sorties et prenait les décisions qui en découlaient, par exemple le traitement du déséquilibre de classes.

6. Réalisation

Cette section résume les étapes ; les paramètres, les commandes et les sorties détaillées figurent dans la documentation technique.

6.1 Données

Le jeu a été récupéré avec TensorFlow Datasets (téléchargement de 827 Mio, 54 303 images) puis exporté en fichiers JPEG rangés par classe. Il a été réparti en trois sous-ensembles stratifiés par classe, avec une graine fixe : 37 995 images pour l'entraînement, 8 129 pour la validation et 8 179 pour le test (70 %, 15 %, 15 %). Les images ne sont pas redimensionnées à ce stade ; la mise à l'échelle à 224 × 224 pixels est faite à la volée, à l'entraînement comme à l'inférence.

L'effectif par classe varie fortement : de 106 images d'entraînement pour `Potato___healthy` à 3 854 pour `Orange___Haunglongbing_(Citrus_greening)`, soit un rapport d'environ 36. Cet écart a motivé deux décisions : pondérer la fonction de perte par classe, et suivre le F1 macro, qui donne le même poids à chaque classe, en plus de l'exactitude. Une augmentation ciblée des classes rares a été écartée, la plus petite classe comptant tout de même 106 images.

Un écart de nommage a été relevé à ce stade : TensorFlow Datasets nomme `Cherry` et `Corn` ce que la liste initiale du code appelait `Cherry_(including_sour)` et `Corn_(maize)`. La liste a été corrigée et son ordre confronté à celui des dossiers, car l'API associe chaque sortie du modèle à une classe par sa position.

6.2 Baseline

Chaque image est décrite par 1 860 valeurs : un histogramme de couleur sur les trois canaux (32 intervalles par canal, soit 96 valeurs) et un histogramme de gradients orientés (HOG, 1 764 valeurs) calculé sur l'image en niveaux de gris réduite à 128 × 128 pixels. Un Random Forest de 200 arbres, pondéré par classe, est entraîné sur ces vecteurs en 108 secondes. Sur le jeu de validation, il atteint 80,74 % d'exactitude et 72,98 % de F1 macro, avec deux classes entièrement manquées (`Apple___Cedar_apple_rust` et `Potato___healthy`).

6.3 Transfert d'apprentissage

Le modèle repose sur MobileNetV2 pré-entraîné sur ImageNet, sans sa couche de classification. Une tête composée d'un average pooling global, d'un dropout à 0,2 et d'une couche dense à 38 sorties (softmax) est ajoutée. Des couches d'augmentation de données (retournement horizontal, rotation, zoom, luminosité) et une couche de mise à l'échelle des pixels précèdent la base. L'ensemble compte 2 306 662 paramètres.

L'entraînement se déroule en deux étapes, sur 16 epochs au total : huit epochs où seule la tête apprend (base gelée, taux d'apprentissage de 1e-3), puis huit epochs où les 30 dernières couches de la base sont dégelées avec un taux de 1e-5. La perte est l'entropie croisée catégorielle, l'optimiseur Adam, la graine 42, et la pondération des classes est calculée sur

l'effectif d'entraînement. L'arrêt anticipé (patience de 5 epochs sur la perte de validation) était actif mais ne s'est pas déclenché. L'entraînement a duré environ une heure et demie sur le GPU du Mac, de 00 h 28 à 01 h 58 le 3 août d'après l'heure de lancement et la date du fichier produit. Sur le jeu de validation, le modèle final atteint 96,48 % d'exactitude et 95,93 % de F1 macro.

6.4 Évaluation

L'évaluation finale porte sur les 8 179 images de test. Elle compare le modèle de transfert à la baseline, ré-entraînée pour l'occasion à partir des caractéristiques déjà extraites, afin que les deux modèles soient jugés sur le même jeu. Le script produit le rapport de classification par classe, la matrice de confusion et la liste des confusions les plus fréquentes. Les résultats sont donnés au chapitre 8.

6.5 API

L'API est écrite avec FastAPI. Elle expose `GET /api/health` (état du service et du modèle) et `POST /api/predict` (réception d'une image, validation du type et de la taille, prétraitement, inférence, classement des classes). Le modèle est chargé une fois au démarrage, suivi d'un appel à vide : sans lui, le premier appel réel prenait plus de 2 s (2,1 et 2,2 s sur deux mesures) ; avec lui, 262 ms. Si aucun modèle n'est présent, l'API démarre en « mode placeholder » : elle renvoie une prédiction factice et déterministe, signalée par le champ `is_placeholder`, ce qui a permis de développer l'interface avant la fin de l'entraînement.

6.6 Interface

L'interface est une application Next.js (App Router, TypeScript) qui suit les maquettes validées par l'auteur. La page d'accueil présente les 14 espèces couvertes. La page de démonstration permet de prendre une photo ou de choisir un fichier, l'envoie à l'API et suit trois étapes (import, analyse, résultat). La page de résultat affiche le diagnostic : classe en français, score de confiance, trois classes les plus probables, fiche descriptive de la maladie (agent pathogène, symptômes, propagation), indication de fiabilité propre à la classe et, le cas échéant, note sur les confusions connues. Les indications de fiabilité et de confusion proviennent de l'évaluation sur jeu de test (section 8.3) et sont enregistrées dans le code de l'interface. L'application s'adapte aux écrans de téléphone, de tablette et d'ordinateur.

6.7 Mise en ligne

L'API est empaquetée dans une image Docker (Python 3.11, dépendances de production, modèle de 21 Mo) puis déployée sur Fly.io, région de Paris, sur une seule machine à processeur partagé et 1 Go de mémoire, avec une vérification de santé sur `/api/health`. L'interface est déployée sur Vercel. Un premier essai d'hébergement sur Render n'a pas abouti, le quota gratuit de l'auteur étant atteint ; le fichier de configuration correspondant est resté dans le dépôt sans avoir été exécuté. Au 9 octobre 2026, l'API et le site répondent.

7. Choix techniques et justifications

Tableau 3. Principaux choix techniques.

Choix	Justification
Python 3.11.6 via pyenv	Version compatible avec TensorFlow 2.18, fixée dans `.python-version` pour que le projet se reconstruise à l'identique. La version installée sur le poste (3.13.9) n'a pas été essayée.
FastAPI	Validation des entrées par les types, documentation OpenAPI générée automatiquement, exécution asynchrone.
MobileNetV2	Recommandé par le cahier des charges pour son compromis précision et taille : 2,3 millions de paramètres, fichier de 21 Mo.
Entraînement en deux étapes	Stabiliser d'abord la tête avec la base gelée évite de dégrader les poids d'ImageNet par des gradients encore mal calibrés ; le taux de 1e-5 du fine-tuning affine ces poids sans les écraser.
Pondération des classes et F1 macro	Rapport d'effectifs de 36 entre classes. Le F1 macro met en évidence les classes ignorées que l'exactitude globale masque.
Normalisation dans le modèle	La couche `Rescaling` est sauvegardée avec le modèle et reste le seul endroit où les pixels sont normalisés ; le prétraitement de l'API se limite au redimensionnement (voir section 9).
Chargement au démarrage et appel à vide	Le coût de chargement de TensorFlow est payé une fois par le serveur, pas par l'utilisateur.
Import différé de TensorFlow	En mode placeholder, TensorFlow n'est pas chargé, ce qui accélère le démarrage.
Modèle versionné dans Git	21 Mo, sous la limite de GitHub : l'image Docker se construit sans étape de téléchargement. Les données (environ 2 Go) ne sont pas versionnées.
Fly.io, une seule machine	Processus actif en continu, adapté à un modèle lent à charger. La seconde machine créée par défaut a été supprimée pour limiter le coût, au prix de la haute disponibilité.
CSS Modules, sans bibliothèque de style	Reproduction directe des maquettes, sans traduction vers un système utilitaire.
Résultat transmis par `sessionStorage`	Aucun stockage côté serveur. L'image est réduite à 900 pixels de côté au plus avant d'être conservée, pour rester sous le quota du navigateur.
Description de la maladie sans traitement	Limite posée par la section 3.2 du cahier des charges (voir section 2.3).

8. Résultats

8.1 Performances sur le jeu de test

Tableau 4. Résultats sur le jeu de test (8 179 images jamais vues à l'entraînement).

Modèle	Exactitude	F1 macro
Baseline (Random Forest, couleur et HOG)	80,95 %	73,58 %
Transfert d'apprentissage (MobileNetV2)	96,47 %	95,66 %
Écart	+15,5 points	+22,1 points

Les valeurs de validation (96,48 % d'exactitude, 95,93 % de F1 macro) et de test (96,47 % et 95,66 %) diffèrent peu : 0,01 point pour l'exactitude et 0,27 point pour le F1 macro. La moyenne pondérée du F1 sur le jeu de test est de 0,96.

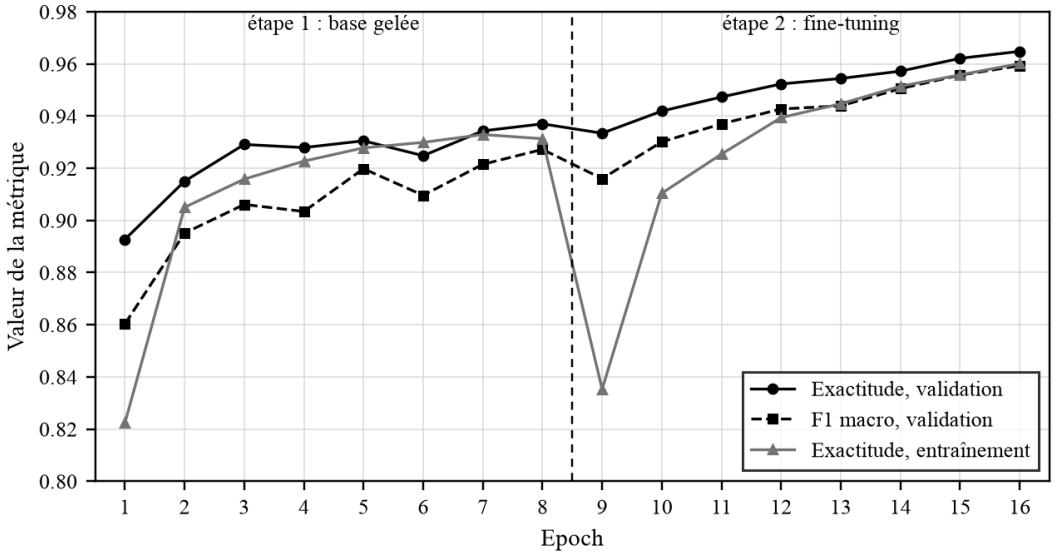


Figure 1. Évolution de l'exactitude et du F1 macro sur la validation, et de l'exactitude sur l'entraînement, au fil des 16 epochs (valeurs relevées dans la sortie du script d'entraînement).

L'exactitude d'entraînement reste inférieure à celle de validation pendant l'essentiel de l'apprentissage : elle est calculée avec l'augmentation de données et le dropout actifs, que l'évaluation sur la validation n'applique pas. La baisse à l'epoch 9 coïncide avec le dégel des 30 couches et le changement de taux d'apprentissage.

8.2 Critères d'acceptation et objectifs

Tableau 5. Critères d'acceptation (section 10 du cahier des charges) et objectifs de taille.

Critère ou objectif	Valeur obtenue	Statut
Exactitude d'au moins 90 % sur le jeu de test	96,47 %	Atteint
F1 macro d'au moins 85 %	95,66 %	Atteint
Dépasser strictement la baseline sur les métriques suivies	+15,5 points d'exactitude, +22,1 points de F1 macro	Atteint
Classification d'une image en moins de 3 s de bout en bout	Temps d'inférence de l'API mesuré de 52 à 262 ms (poste local, conteneur, hébergement) ; temps de bout en bout non chronométré	Non vérifié
Interface conforme au design system sur tous les points de rupture	Contrôle visuel à 375, 768 et 1 280 px de large ; pas d'audit au pixel près	Vérifié en partie
Chargement initial de l'interface inférieur à 1,5 s	Non mesuré	Non vérifié
Documentation permettant à un tiers de réentraîner le modèle sans assistance	Documentation technique fournie ; non éprouvée par un tiers	Non vérifié

Critère ou objectif	Valeur obtenue	Statut
Modèle de moins de 20 Mo après conversion (objectif indicatif)	Fichier Keras de 21 Mo ; conversion TensorFlow Lite non réalisée	Non vérifié

8.3 Analyse des erreurs

La matrice de confusion (figure 2) présente une diagonale nette. Les erreurs se concentrent sur des maladies aux symptômes visuellement proches, surtout chez la tomate. Le tableau 6 donne les confusions les plus fréquentes.

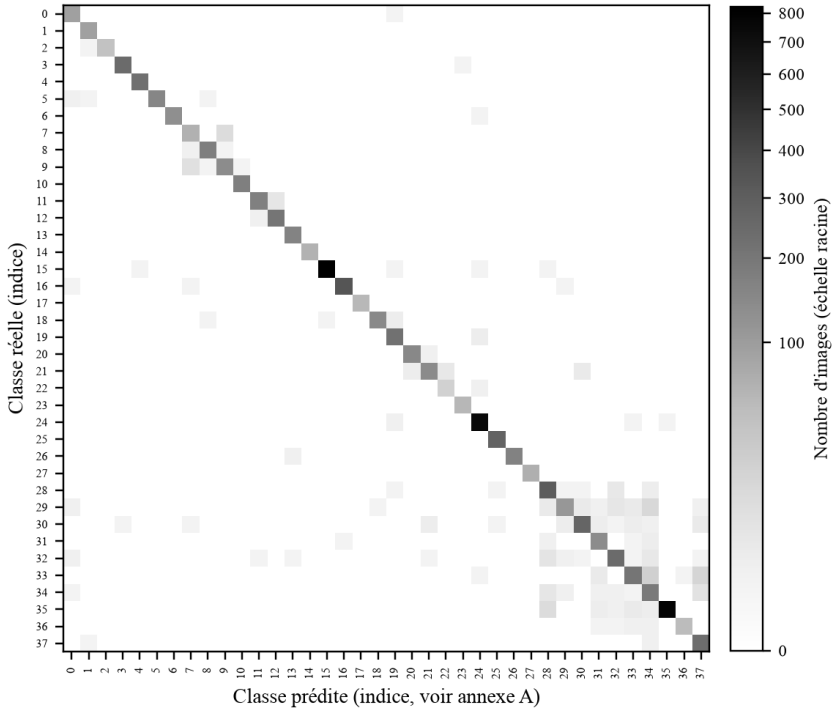


Figure 2. Matrice de confusion du modèle de transfert sur le jeu de test (lignes : classe réelle ; colonnes : classe prédite ; échelle de gris en racine carrée du nombre d'images). La correspondance entre indices et classes figure en annexe A de la documentation technique.

Tableau 6. Huit confusions les plus fréquentes sur le jeu de test (extrait du rapport d'évaluation).

Classe réelle	Prédite à tort comme	Cas
`Tomato___Spider_mites Two-spotted_spider_mite`	`Tomato___Target_Spot`	23
`Tomato___Spider_mites Two-spotted_spider_mite`	`Tomato___healthy`	19
`Tomato___Early_blight`	`Tomato___Target_Spot`	16
`Corn___Cercospora_leaf_spot Gray_leaf_spot`	`Corn___Northern_Leaf_Blight`	13
`Tomato___Tomato_Yellow_Leaf_Curl_Virus`	`Tomato___Bacterial_spot`	13

Classe réelle	Prédite à tort comme	Cas
Corn___Northern_Leaf_Blight	Corn___Cercospora_leaf_spot Gray_leaf_spot	10
Tomato___Target_Spot	Tomato___healthy	8
Tomato___Septoria_leaf_spot	Tomato___Bacterial_spot	7

Tableau 7. Classes aux plus faibles précision ou rappel.

Classe	Précision	Rappel	Images de test
Tomato___Target_Spot	0,76	0,90	212
Potato___healthy	0,81	0,92	24
Corn___Cercospora_leaf_spot Gray_leaf_spot	0,82	0,83	78
Tomato___healthy	0,87	0,99	240
Tomato___Spider_mites Two-spotted_spider_mite	0,92	0,81	252
Tomato___Early_blight	0,92	0,73	150

La précision la plus basse (0,76) concerne `Tomato___Target_Spot` ; le rappel le plus bas (0,73) concerne `Tomato___Early_blight`. Les deux classes rares que la baseline manquait entièrement sur le jeu de validation sont reconnues : `Apple___Cedar_apple_rust` (précision 1,00, rappel 0,98) et `Potato___healthy` (précision 0,81, rappel 0,92). Cette dernière ne compte que 24 images de test, ce qui rend ses chiffres peu stables.

Ces valeurs alimentent l'indication de fiabilité affichée par l'interface : « excellente » à partir de 0,97 de précision, « bonne » à partir de 0,90, « moyenne » en dessous.

8.4 État au 9 octobre 2026

- API de production : `GET /api/health` renvoie un statut `ok`, `model_loaded` à vrai, `is_placeholder` à faux et 38 classes.
- Site de production : réponse HTTP 200. Un parcours complet (image d'exemple, appel à l'API, page de résultat) a été exécuté sans erreur le 3 août 2026.
- Tests automatisés du backend : 5 réussis sur 5 (exécution du 9 octobre 2026).
- Interface : ESLint et vérification des types TypeScript sans erreur (exécution du 9 octobre 2026).

9. Difficultés rencontrées et solutions

Tableau 8. Difficultés rencontrées, causes et solutions.

Difficulté	Cause identifiée	Solution
Échec du téléchargement des données	Conflit entre la dernière version de `tensorflow-metadata` (fichiers protobuf générés avec une version 6) et la contrainte `protobuf` inférieure à 6 de TensorFlow ; paquet `importlib_resources` absent ; script ne trouvant pas le package `app`.	`tensorflow-metadata` épinglé à 1.16.1, paquet installé, dossier backend ajouté au chemin d'import.
Classes mal nommées	Noms de `Cherry` et `Corn` différents entre la liste du code et TensorFlow Datasets.	Correction de `labels.py` ; contrôle d'égalité avec les dossiers, repris dans le script d'entraînement.
Tests en échec, puis fragiles	Le chargement du modèle n'était pas déclenché sans gestionnaire de contexte ; ensuite, assertions supposant l'absence de modèle entraîné.	Fixture `with TestClient(app)` ; assertions sur des invariants (`model_loaded` est l'inverse de `is_placeholder`).
Prédictions fausses et très confiantes via l'API (feuille de tomate saine classée « maïs sain » à 99,88 %)	Double normalisation : le prétraitement de l'API normalisait les pixels, puis la couche `Rescaling` du modèle recommençait.	Normalisation retirée du prétraitement ; contrôle sur quatre appels, dont deux sur des classes rares.
Premier appel lent (plus de 2 s)	Compilation du graphe TensorFlow au premier appel.	Appel à vide au démarrage ; premier appel ensuite mesuré à 262 ms.
Espace disque limité (12 à 18 Go libres)	Données brutes, copies du découpage, modèle Random Forest sérialisé.	Suppression de `data/raw` (1,6 Go), des points de contrôle (31 Mo) et du Random Forest sérialisé (1,2 Go), tous régénérables.
Hébergement sur Render impossible	Quota gratuit atteint.	Passage à Fly.io, qui exige une carte bancaire enregistrée.
Arrêt de l'API au démarrage en production	`pydantic-settings` décode en JSON les variables d'environnement destinées à un champ de type `tuple`, avant tout validateur : une liste séparée par des virgules provoquait une `SettingsError`. Le défaut n'apparaissait pas en local, où la valeur par défaut était utilisée.	Champ typé `str`, découpage dans une propriété. Vérifié en local, en conteneur puis en production ; la machine avait atteint sa limite de 10 redémarrages avant la correction.
Coût doublé sur Fly.io	Deux machines créées par défaut.	Réduction à une machine (`fly scale count 1`).
Débordements en affichage mobile	Photos de l'accueil disposées pour un grand écran ; fil d'étapes trop large sous 480 px.	Rangée défilante sous 640 px ; fil réduit à des points sous 480 px.

Difficulté	Cause identifiée	Solution
Erreurs ESLint (règle React `set-state-in-effect`)	État initialisé dans des effets.	Lecture de `prefers-reduced-motion` et de `sessionStorage` par `useSyncExternalStore`.

10. Limites du travail

- **Conditions de laboratoire.** PlantVillage montre des feuilles isolées sur fond neutre. Aucun essai sur des photos de terrain n'a été fait, alors que le cahier des charges le recommande : les 96,47 % ne s'appliquent pas à ce cas.
- **Entrées hors sujet.** Le modèle répartit toujours sa probabilité entre les 38 classes. Une photo qui n'est pas une feuille reçoit donc quand même une classe ; le seuil de 70 % limite le risque sans le supprimer. Ce comportement découle de la conception du modèle ; il n'a pas été testé avec des images hors sujet.
- **Classes ambiguës.** Les maladies voisines, surtout chez la tomate, sont confondues dans une proportion non négligeable ; la précision descend à 0,76 pour une classe.
- **Une seule évaluation.** Un seul découpage et une seule graine : pas de validation croisée, pas d'intervalle de confiance.
- **Pas de traitement proposé.** C'est volontaire (section 2.3), mais le diagnostic s'arrête à la description de la maladie.
- **Usage mobile non traité.** Pas de conversion TensorFlow Lite ; la taille cible n'est pas vérifiée.
- **Exploitation.** Inférence synchrone, pas de test de charge, une seule machine sans redondance.
- **Interface.** Aucun test automatisé, temps de chargement non mesuré. La commande `npm audit` signale trois vulnérabilités de niveau « high » dans des dépendances internes de Next.js (postcss, sharp) ; elles n'ont pas été traitées, la correction automatique proposée imposant une rétrogradation vers une version majeure très ancienne de Next.js.
- **Données de référence écrites dans le code.** Les fiches maladies, les traductions et les valeurs de fiabilité sont codées en dur dans l'interface et à mettre à jour à la main si le modèle est ré-entraîné.
- **Redimensionnement.** Les redimensionnements bilinéaires de l'entraînement (TensorFlow) et de l'inférence (Pillow) ne sont pas strictement identiques ; l'écart n'a pas été quantifié.
- **Licence des images.** Le cahier des charges indique un usage académique et de recherche pour PlantVillage ; les conditions précises d'affichage de ses images sur un site public n'ont pas été vérifiées.

11. Perspectives d'amélioration

- Tester le modèle sur un petit échantillon de photos de terrain et documenter l'écart, comme le prévoit le cahier des charges.
- Chronométrer le parcours de bout en bout et le chargement initial, puis convertir le modèle en TensorFlow Lite pour vérifier sa taille.
- Comparer avec EfficientNetB0, ce qui suppose d'adapter la normalisation à l'entraînement et à l'inférence.
- Ajouter une détection des entrées hors sujet, ou un refus de répondre sous un seuil de confiance.
- Traiter les classes à faible rappel (`Tomato___Early_blight`) par une augmentation ciblée ou un échantillonnage rééquilibré.
- Reprendre les évolutions listées en section 12 du cahier des charges : localisation de la zone atteinte, extension à d'autres espèces, module de conseil construit avec un expert agronome.
- Ajouter des tests automatisés à l'interface, mesurer ses performances et mettre en place une intégration continue.

12. Conclusion

Le projet livre une chaîne complète, de la donnée brute au service en ligne : un modèle de transfert d'apprentissage dont l'exactitude (96,47 %) et le F1 macro (95,66 %) sur jeu de test dépassent les seuils du cahier des charges et la baseline de plus de 15 points, une API déployée, et une interface qui accompagne le diagnostic d'explications. Parmi les huit critères et objectifs du tableau 5, trois sont atteints, un est vérifié en partie et quatre ne sont pas vérifiés (temps de bout en bout, chargement initial, reprise par un tiers, taille après conversion).

Sur le plan méthodologique, les deux défauts les plus coûteux, la normalisation appliquée deux fois et la lecture de la configuration en production, ne produisaient aucune erreur visible pendant le développement. Ils n'ont été trouvés qu'en interrogeant l'API avec le vrai modèle, puis en déployant. Les performances annoncées restent mesurées sur des images de laboratoire ; leur transposition à des photos prises au champ n'est pas démontrée.